

N-ary Tree Postorder Traversal

Difficulty: Easy

Patterns: Tree, Depth-First Search, Stack, Recursion

LeetCode: <https://leetcode.com/problems/n-ary-tree-postorder-traversal/>

1. Problem Summary

Given the `root` of an N-ary tree, return the postorder traversal of its node values.

For an N-ary tree, postorder means:

1. Visit all children from left to right.
2. Append the current node after all of its children are finished.

Examples:

- `root = [1,null,3,2,4,null,5,6]` returns `[5,6,3,2,4,1]`
- `root = []` returns `[]`

2. Constraints and Observations

- Constraint: The number of nodes is in `[0, 10^4]`.
- What it implies: Any accepted solution should be linear, $O(n)$, because every node may need to appear in the answer.
- Constraint: `0 <= Node.val <= 10^4`.
- What it implies: Values do not affect the traversal logic. We only care about tree shape and child order.
- Constraint: The height of the N-ary tree is at most `1000`.
- What it implies: Recursion is conceptually natural, but an iterative solution is often preferred for the follow-up because it avoids depending on call stack behavior.
- Observation: In postorder, a parent is appended only after every child subtree has been fully processed.

3. Pattern Recognition

This is a tree DFS traversal problem.

The key difference from binary-tree postorder is that each node has a list of children instead of just `left` and `right`. The order still matters: children must be processed from left to right.

The main patterns are:

- Recursion: let the call stack remember where we are in the tree.
- Explicit stack: simulate DFS manually.
- Reverse traversal trick: generate `root, right-to-left children` order and reverse it to get `left-to-right children, root`.

4. Naive Approach

The first natural idea is recursive DFS:

1. If the node is `None`, stop.
2. Recursively postorder every child.
3. Append the current node.

This is already optimal in time because it visits every node once. The only reason to look for iterative variants is the follow-up: "Recursive solution is trivial, could you do it iteratively?"

So the bottleneck is not time complexity. The bottleneck is dependency on recursion stack space and whether we can simulate the same order with explicit stacks.

5. Key Intuition

Postorder wants:

```
children first, parent last
```

If we use a stack and append values as soon as we pop nodes, we naturally get a preorder-like order:

```
parent first, children later
```

That sounds wrong, but there is a useful symmetry:

```
postorder:      left children ... right children ... root
reverse of that: root ... right children ... left children
```

So if an iterative traversal builds `root, right-to-left subtrees`, reversing the result gives the real postorder.

6. Optimal Solutions Ranked

All meaningful approaches are $O(n)$ time. No solution can do better than $O(n)$ because the output itself contains one value per node.

Variant 1: Recursion

- Idea: DFS each child first, append the node afterward.
- Time: $O(n)$.
- Space: $O(h)$ call stack, where h is tree height. The answer uses $O(n)$.
- Tradeoff: Cleanest and easiest to explain, but relies on recursion.

Variant 2: Two Stacks

- Idea: Use `stack1` to visit nodes and `stack2` to reverse the append order. Pop from `stack2` at the end.
- Time: $O(n)$.
- Space: $O(n)$ extra stack space.
- Tradeoff: Very explicit and interview-friendly, but uses two stacks.

Variant 3: One Stack With Reverse Result

- Idea: Pop a node, append its value immediately, push its children left-to-right, then reverse the result.
- Time: $O(n)$.
- Space: $O(n)$ for the explicit stack plus output.
- Tradeoff: Shortest iterative solution. The only trick is understanding why reversing fixes the order.

Variant 4: One Stack With Visited Marker

- Idea: Push `(node, visited)` frames. The first time we see a node, push it back as visited, then push its children in reverse order. The second time, append it.
- Time: $O(n)$.
- Space: $O(n)$ in the worst case.
- Tradeoff: Simulates recursion directly without reversing the final list. Slightly more code, but the order is very transparent.

Most practical final choice: one stack with reverse result. It is iterative, compact, and accepted for the follow-up while preserving $O(n)$ time.

7. Most Optimal Approach

Use a stack to produce the reverse of postorder, then reverse the answer.

For each popped node:

1. Append its value to `result`.
2. Push its children from left to right.

Because a stack is LIFO, the rightmost child is processed first. This creates:

```
root, right subtree, ..., left subtree
```

Reversing that order gives:

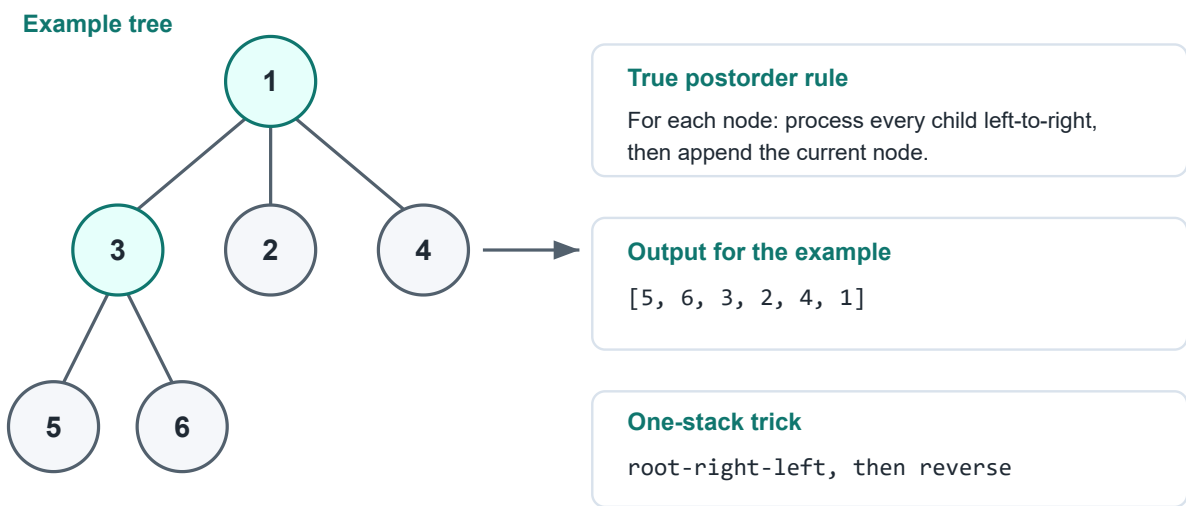
```
left subtree, ..., right subtree, root
```

That is exactly N-ary postorder.

8. Algorithm

1. If `root` is `None`, return an empty list.
2. Initialize `stack = [root]` and `result = []`.
3. While the stack is not empty:
 1. Pop a node.
 2. Append `node.val` to `result`.
 3. Push every child in `node.children` from left to right.
4. Reverse `result`.
5. Return `result`.

9. Visual Explanation



Caption: Postorder processes all children before the parent. The one-stack trick builds the reverse order first, then flips it.

For the tree:



The correct postorder is:

[5, 6, 3, 2, 4, 1]

10. Dry Run

Using the one-stack reverse-result method:

Step	Pop	Append result before final reverse	Children pushed left-to-right	Stack after push
1	1	[1]	3, 2, 4	[3, 2, 4]
2	4	[1, 4]	none	[3, 2]

Step	Pop	Append result before final reverse	Children pushed left-to-right	Stack after push
3	2	[1, 4, 2]	none	[3]
4	3	[1, 4, 2, 3]	5, 6	[5, 6]
5	6	[1, 4, 2, 3, 6]	none	[5]
6	5	[1, 4, 2, 3, 6, 5]	none	[]

Before reversing:

```
[1, 4, 2, 3, 6, 5]
```

After reversing:

```
[5, 6, 3, 2, 4, 1]
```

11. Correctness Argument

Invariant for the one-stack reverse-result approach:

At every step, `result` stores nodes in the order they are removed by a DFS that visits the current node before its children and processes later siblings before earlier siblings.

Why this works:

- The current node is appended before its children, so the order is parent-before-descendants.
- Children are pushed left-to-right, so because the stack is LIFO, the rightmost child subtree is processed before the leftmost child subtree.
- Therefore, the collected order is the reverse of N-ary postorder: `root, right-to-left subtrees`.
- Reversing the collected list gives `left-to-right subtrees, root`, which is exactly postorder.

Because every node is pushed once, popped once, and appended once, every node appears exactly once in the final traversal.

12. Complexity Analysis

For all approaches:

- Time: $O(n)$, where n is the number of nodes.
- Output space: $O(n)$ for the returned list.

Auxiliary space by approach:

- Recursion: $O(h)$ call stack, where h is the tree height.
- Two stacks: $O(n)$ in the worst case.
- One stack with reverse result: $O(n)$ in the worst case.
- One stack with visited marker: $O(n)$ in the worst case.

No approach can beat $O(n)$ time because the result contains n values.

13. Edge Cases

- Empty tree: `root = None`, return `[]`.
- Single node: return `[root.val]`.
- Root with many leaf children: children must appear left-to-right, then root.
- Deep chain: recursion may use a deep call stack; iterative versions avoid that concern.
- Uneven N-ary tree: stack methods still work because they depend only on child order, not balanced shape.

14. Approach Code: Recursion

```
from typing import List, Optional

"""
# Definition for a Node.
class Node:
    def __init__(self, val: Optional[int] = None, children: Optional[List['Node']] = None):
        self.val = val
        self.children = children
"""

class Solution:
    def postorder(self, root: 'Node') -> List[int]:
        result = []

        def dfs(node: 'Node') -> None:
            if node is None:
                return

            for child in node.children:
                dfs(child)

            result.append(node.val)

        dfs(root)
        return result
```

15. Approach Code: Two Stacks

```
from typing import List, Optional

"""
# Definition for a Node.
class Node:
    def __init__(self, val: Optional[int] = None, children: Optional[List['Node']] = None):
        self.val = val
        self.children = children
"""

class Solution:
    def postorder(self, root: 'Node') -> List[int]:
        if root is None:
            return []

        stack1 = [root]
        stack2 = []

        while stack1:
            node = stack1.pop()
            stack2.append(node)

            for child in node.children:
                stack1.append(child)

        result = []
        while stack2:
            result.append(stack2.pop().val)

        return result
```

16. Approach Code: One Stack With Visited Marker

```
from typing import List, Optional

"""
# Definition for a Node.
class Node:
    def __init__(self, val: Optional[int] = None, children: Optional[List['Node']] = None):
        self.val = val
        self.children = children
"""

class Solution:
    def postorder(self, root: 'Node') -> List[int]:
        if root is None:
            return []

        result = []
        stack = [(root, False)]

        while stack:
            node, visited = stack.pop()

            if visited:
                result.append(node.val)
                continue

            stack.append((node, True))

            for child in reversed(node.children):
                stack.append((child, False))

        return result
```

17. Final Clean Code

```
from typing import List, Optional

"""
# Definition for a Node.
class Node:
    def __init__(self, val: Optional[int] = None, children: Optional[List['Node']] = None):
        self.val = val
        self.children = children
"""

class Solution:
    def postorder(self, root: 'Node') -> List[int]:
        if root is None:
            return []

        result = []
        stack = [root]

        while stack:
            node = stack.pop()
            result.append(node.val)

            for child in node.children:
                stack.append(child)

        return result[::-1]
```

18. Key Takeaways

- Main pattern: DFS traversal on an N-ary tree.
- Main trick: Postorder is the reverse of root, right-to-left subtree order.
- Best interview explanation: "I collect reverse postorder with one stack, then reverse the list."
- Recursion is the clearest explanation, but the one-stack method is the cleanest iterative follow-up answer.